

ECHO: Dyadic Event-Stream Intensities for Temporal Knowledge Graph Forecasting

Authors to be added

Computer Network Information Center, Chinese Academy of Sciences
University of Chinese Academy of Sciences
Beijing, China

Abstract

Temporal knowledge graph forecasting asks which object completes a query $(s, r, ?, t)$ given every fact before t . The strongest published models answer it by reading the history of the query pair (s, r) : a copy mechanism over its past objects, or a graph built from its past facts. We show that this is the wrong unit of evidence. The unit that carries the signal is the *dyad*, the ordered entity pair (s, o) together with the typed, timestamped stream of every event between them under any relation and in either direction. On ICEWS18, 23% of test queries have an answer that never occurred with (s, r) yet had interacted with s before; every copy mechanism is silent on them by construction. A counter that merely reads the dyad, with no trained parameter, already matches DaeMon on that benchmark.

We present ECHO, which treats a query as a draw from a superposition of four marked point processes whose intensities are added, never gated: a structural intensity from snapshot evolution, a popularity intensity over all entities, a dyadic intensity in which a small Transformer encodes the dyad’s event stream, and a path intensity that forwards each dyadic state along the candidate’s own most recent typed edges. Candidates of one query attend to one another before they are scored, so an intensity can depend on what the competing candidates’ streams look like.

Over three seeds and five benchmarks, ECHO sets the best published MRR and H@1 on YAGO (91.82 / 90.57) and WIKI (83.41 / 80.44), the best H@1 on GDEL (18.16), and on ICEWS18 and ICEWS14s exceeds every model whose code we could run under one protocol. A stratified evaluation, defined from the data alone, shows where each intensity acts: the dyadic stream and its relation types matter on event data and not on yearly data, exactly where the dyad stratum exists. Code, result files and the diagnostic are released.

1 Introduction

A temporal knowledge graph (TKG) records facts as quadruples (s, r, o, t) : a subject s and an object o linked by

relation r at time t . Forecasting on a TKG means completing $(s, r, ?, t)$ from the facts strictly before t . The benchmarks are political event streams (ICEWS, GDEL) and yearly encyclopaedic graphs (YAGO, WIKI), and the task has drawn a dense line of work over six years: recurrent graph encoders [15, 22, 20, 5], copy mechanisms [33, 29], path and rule reasoners [12, 26, 24, 4, 6, 10], global history graphs [19, 2, 31, 3] and, most recently, diffusion models [1, 7, 8].

A recurring finding runs through this literature and is easy to underrate. A baseline that only looks up which objects recurred for (s, r) in the past, with a handful of hand-set parameters, is competitive with or better than most published models on three of five benchmarks [9]. A counter with no temporal information at all reaches H@10 above 0.9 on some of them [30]. Over 80% of ICEWS test events have happened before [14]. Progress, in other words, is measured on a task whose majority is repetition, and the methods that win have in common that they read the history of the query pair (s, r) very well.

The unit of evidence. This paper starts from a question the copy family cannot ask. For a query $(s, r, ?, t)$, what if every past event between s and a candidate o were allowed to speak, under any relation and in either direction, with its type and its time? We call the ordered pair (s, o) with that stream a *dyad* (Fig. 1). The question has an answer that is a property of the data, not of any model (Section 4). On ICEWS18 the test queries split into three disjoint strata: 50.6% whose answer already occurred with (s, r) , 23.4% whose answer never did but had interacted with s under some other relation, and 26.0% whose answer never interacted with s at all. The second stratum is invisible to a copy mechanism by construction. A hand-built counter that reads the dyad instead of the triple, with the same time decay and no training, lifts the untrained score on ICEWS18 from MRR 28.0 to 31.8, which is the level of DaeMon [4], and on ICEWS14 from 37.1 to 43.3, which is the level of TLogic [24].

The model. ECHO is built on that measurement. A query is a draw from a superposition of marked point processes, and the intensities add (Section 5). The dyadic

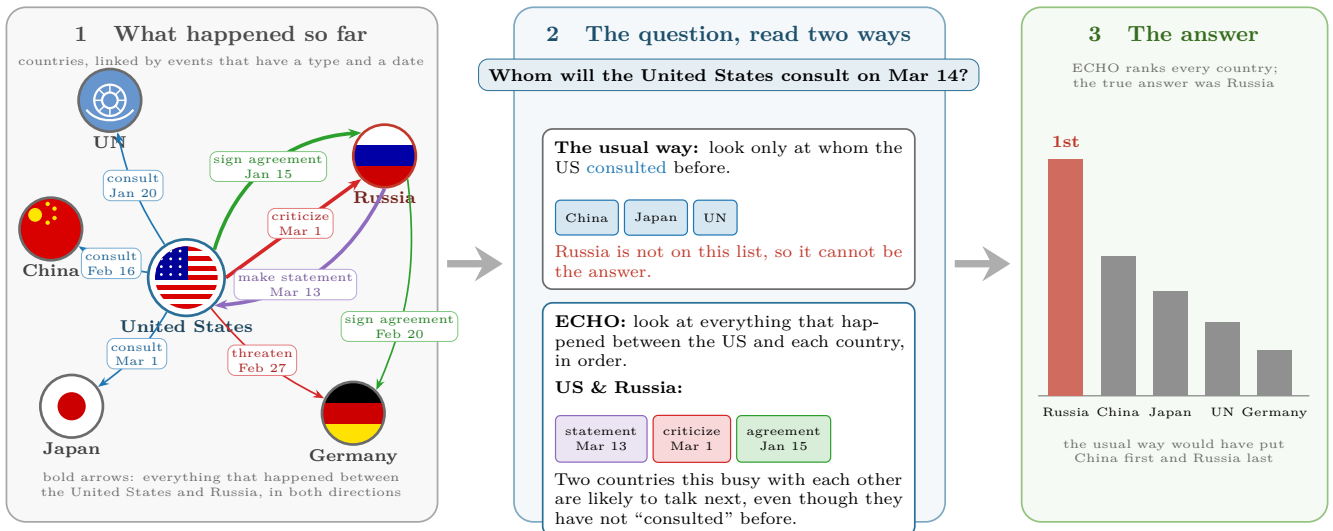


Figure 1: What ECHO does, on an example. *Left:* a temporal knowledge graph, countries linked by events that carry a type and a date. *Middle:* the forecasting question and two ways of reading the past. The usual way looks only at whom the United States *consulted* before, and Russia is not on that list. ECHO reads everything that happened between the United States and each country, in order and with its timing; the stream for Russia has no “consult” in it, but its recent agreement, criticism and statement say the two are in close contact. *Right:* ECHO ranks Russia first; the usual way would rank it last. On ICEWS18 the answer is visible only in this second reading for 23.4% of test queries (Section 4). Names and dates are illustrative.

intensity encodes the last L events of the dyad as a sequence of (relation, elapsed time) tokens with a two-layer Transformer, so that recurrence, cross-relation excitation, reciprocity and order all enter through one mechanism and nothing is confined to decay monotonically in time. Because a dyad is only instantiated when it has history, the per-pair point process that earlier work dismissed as combinatorially infeasible [13] costs a few dozen sequences per query. Two further intensities extend the reach of the dyad: a path intensity forwards each dyadic state along the candidate’s own most recent typed edges, reaching the 54% of “cold” answers that are partners of a partner, and a popularity intensity scores every entity from multi-scale decayed counts. A structural intensity from snapshot evolution, which is prior work [22, 5], completes the superposition. Finally, the candidates of one query attend to one another before they are scored, so the intensity of a candidate can depend on what the other candidates’ streams look like.

What we claim, and what we do not. Over three seeds and five benchmarks (Section 6), ECHO reports the best published MRR and H@1 on YAGO and WIKI, the best H@1 on GDELT, and on ICEWS18 and ICEWS14s it is above every model whose code we could run under a single protocol, including LogCL [2], which we re-ran. Three recent methods report higher ICEWS numbers [31, 18, 17]; none has released code, and evaluation settings in this field are known to move results by several points [11]. We therefore state the comparison exactly as it is rather than pool numbers across protocols. The contribution

we stand on is not a leaderboard margin but a mechanism and a measurement: the dyad stratum exists, it is large on event data and empty on yearly data, and the stratified evaluation in Section 7 shows the dyadic stream and its relation types act precisely where that stratum is (ICEWS18: +0.58 and +0.43 MRR) and not where it is not (YAGO: 0.00 and −0.03).

Contributions.

- A data-only diagnostic that partitions test queries into recurrent, dyad-only and cold strata, and the measurement that the dyad-only stratum is a quarter of ICEWS queries and that an untrained dyad counter matches DaeMon.
- ECHO, a superposition of four intensities built around a Transformer over the dyad’s typed event stream, with a path intensity that composes a dyadic state with a typed edge and a competition layer across the candidates of a query.
- Three-seed results on five benchmarks, ablations on an event and a yearly dataset, and a stratified analysis that attributes each intensity to the stratum it was built for. All code, the diagnostic and every result file are released.

2 Related Work

Recurrence and copy mechanisms. CyGNet [33] introduced a copy mode that scores the objects that already

answered the same (s, r) in the past, next to a generation mode over the whole vocabulary, on the observation that many facts recur. CENET [29] learns whether a query follows its historical or its non-historical dependency and applies the decision as a mask at inference. TiRGN [19] pairs a local recurrent graph encoder with a global history encoder for repeated facts and a time-guided decoder with periodicity. Gastinger et al. [9] showed that a non-learned recurrency baseline is competitive with eleven published models on three of five benchmarks, and CountTRuCoLa [10] turned that observation into four rule types with confidence functions that combine recency and frequency. All of these define “history” as the past of the query pair (s, r) . The dyad-only stratum of Section 4, where the answer interacted with s under another relation, lies outside that definition by construction.

Snapshot evolution. RE-NET [15] and RE-GCN [22] evolve entity representations through a sequence of snapshot graphs, with a relational GCN per snapshot and a recurrent unit across them; CEN [20] adds a length-aware CNN with curriculum learning and an online setting; DiM-Net [5] adds cross-time links at equal hop depth and a disentanglement of active and stable node features. HiS-Match [21] frames the task as matching the query’s historical structure against each candidate’s. Our structural intensity is this family and is claimed as nothing more than the backbone; trained alone it reaches RE-GCN’s level on ICEWS18 (Section 7).

Global history graphs. The strongest recent results come from models that build a query-specific graph from the whole history and run message passing over it. LogCL [2] samples the one-hop facts of the query subject and the one-hop facts of the objects that answered (s, r) before, applies entity-aware attention, and regularises the fusion of local and global views with four contrast patterns. HisRES [31] aggregates all historical facts of the query pair into a global relevance graph and merges it with a multi-granularity recent encoder; its own ablation attributes six MRR points on ICEWS18 to that graph. CID-TKG and CHE-TKG [18, 17] add a second graph of recent facts retrieved through temporal logical rules and align the two views contrastively. CognTKE [3] runs a fast one-hop and a slow multi-hop reasoner over a relation digraph. ECHO reads the same facts as event streams rather than as a graph: the stream encoder sees the order and the types of a dyad’s events, the path intensity composes a stream with a typed edge, and nothing is contextualised by a second GNN. Of the three strongest members of this family, none has released code; the comparison in Section 6 says so explicitly.

Paths, rules and explanation. xERTE [12] expands a query-centred subgraph with temporal attention; TITer [26] walks it with a reinforcement-learning agent;

DaeMon [4] carries a path memory between the query subject and every candidate without entity representations, and TiPNN [6] reasons over query-aware temporal paths on a unified history graph. TLogic [24] extracts temporal logical rules by temporal random walks. Our untrained typed-dyad counter behaves like TLogic’s length-one rules and reaches its ICEWS14 score; the path intensity is a learned form of its length-two rules, with the rule body read from an event stream instead of a single past fact.

Point processes. Know-Evolve [28] modelled each fact as a multivariate point process whose intensity is a function of evolving entity embeddings. GHNN [13] extended the neural Hawkes process to graph sequences and argued that treating every possible link as its own event-type dimension is infeasible because the number of types explodes. GAttNHP [27] encodes subject–relation chains as continuous-time point processes with grouped Hawkes priors. ECHO keeps the point-process reading and resolves the combinatorial objection: a dyad is instantiated only when it has history, so the per-pair process costs a few dozen short sequences per query, and its kernel is a learned function of a sinusoidal time encoding rather than a decay.

Diffusion models. DiffuTKG [1] casts the prediction as denoising the target fact with a Transformer conditioned on the query’s history; NADEX [7] adds batch-wise negative prototypes to the conditioning and FreqDiff [8] a spectral calibration. These are orthogonal to the evidence question we study and are included in the comparison with their own reported numbers.

Evaluation. Gastinger et al. [11] documented that filter settings, prediction horizon and dataset versions move TKG forecasting results by several points and distort comparisons; the recurrency baseline [9], the co-occurrence shortcut of Zhang et al. [30], strikingness-aware reweighting [14] and the synthetic shift benchmark of Özdemir et al. [25] all say that aggregate metrics on these benchmarks are dominated by repetition. The stratified evaluation of Section 7 is the real-data counterpart of probing mechanisms separately, and it is why we re-ran the one strong competitor with code under our protocol instead of quoting it.

Different settings. Inductive reasoning over emerging entities [32, 16] and continual fine-tuning at test time [23] report gains under their own protocols. Our cold strata are not the emerging-entity problem: both entities are known, they have simply never interacted.

3 Preliminaries

Task. A temporal knowledge graph is a set of quadruples $\mathcal{G} = \{(s, r, o, t)\}$ over entities $\mathcal{E}$ ($|\mathcal{E}| = N$), relations $\mathcal{R}$ and

discrete timestamps $t \in \{0, \dots, T - 1\}$. We re-index time to the rank of the timestamp on the full timeline, so that one step is one snapshot on every dataset whatever the raw unit (a day on ICEWS, fifteen minutes on GDEL, a year on YAGO and WIKI). Following every method we compare with, each fact is augmented with its inverse (o, r^{-1}, s, t) , giving $2|\mathcal{R}|$ relation types, and a query is $(s, r, ?, t)$ with r ranging over both directions. The model ranks all N entities as the object using only facts with timestamp strictly less than t ; at test time the true history before t is available, which is the setting of RE-GCN and its successors [22].

Protocol. We report MRR and Hits@ k under the time-aware filter: for a query $(s, r, ?, t)$, the other true objects of (s, r) at time t are removed from the ranking and nothing else is. Ties are resolved to their average rank, $1 + \#\{\text{better}\} + (\#\{\text{tied}\} - 1)/2$. The optimistic rule that counts only strictly better scores credits a model with rank 1 whenever thousands of entities share the target’s score, which a copy-style model produces by default; we found it inflates H@10 on YAGO to 99.9 against a published ceiling of 93.3, and we do not use it. Model selection is on validation MRR; test is evaluated once with the selected checkpoint.

Dyads and strata. For a query $(s, r, ?, t)$ and a candidate o , the *dyad stream* $\mathcal{D}_{so}(t)$ is the time-ordered sequence of all events (ρ, τ) with $\tau < t$ such that $(s, \rho, o, \tau) \in \mathcal{G}$, where ρ ranges over both directions, so an event in which o acted on s appears with the inverse type. The *triple history* is the sub-stream with $\rho = r$. These two sets induce a partition of the test queries that is defined from the data alone:

Definition 1 (Strata). *A test query with answer o^* is **recurrent** if $(s, r, o^*, \tau) \in \mathcal{G}$ for some $\tau < t$; **dyad-only** if it is not recurrent but $\mathcal{D}_{so^*}(t)$ is non-empty; and **cold** otherwise. A recurrent query is further **blocked** if some $o' \neq o^*$ with triple history is both at least as recent and at least as frequent as o^* , and **clean** otherwise. A cold query is **cold-2hop** if o^* is a partner of one of the P_1 most recent partners of s , and **cold-far** otherwise.*

The blocked stratum is the set no scorer of the form $f(\text{count}) \cdot g(\Delta t)$ with f non-decreasing and g non-increasing can order correctly, whatever its parameters: the dominating distractor scores at least as high on both arguments. Every copy mechanism we know of is of that form. The dyad-only stratum is the set where such a scorer has no input at all.

4 What the Dyad Knows: A Data-Only Diagnostic

Before any model is trained we ask two questions of the data. How large are the strata of Definition 1? And

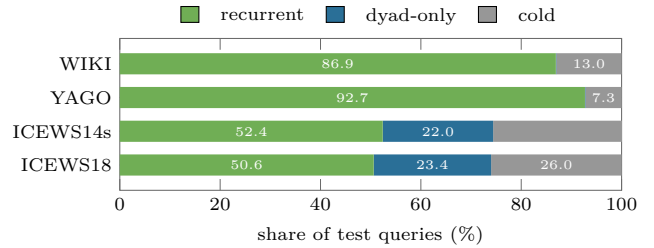


Figure 2: Strata are a property of the data. Share of test queries whose answer is recurrent (occurred with (s, r) before), dyad-only (never with (s, r) , but s and the answer interacted under another relation or direction) and cold (never interacted). A copy mechanism is silent on the dyad-only stratum by construction. It is a quarter of the ICEWS queries and absent from YAGO and WIKI.

does reading the dyad instead of the triple carry ranking information? Both are answered with hand-built counters that share one time decay $w(\Delta) = e^{-0.3\Delta} + 0.2e^{-0.02\Delta} + 0.02$ (in snapshots) and read only events before the query time. No parameter is fitted to the test set; the one learned quantity, a relation-to-relation confidence, is counted on the training split.

- **Recurrence** scores o by $\sum w(\Delta)$ over past (s, r, o) events. This is the copy family and the strict recurrence baseline of [9].
- **Blind dyad** scores o by $\sum w(\Delta)$ over every past event between s and o , relation ignored.
- **Typed dyad** weights each of those events by $\text{conf}(\rho \rightarrow r)$, the training-split frequency with which a ρ -event on a dyad is followed by an r -event within seven steps. This is a length-one temporal rule in the sense of TLogic [24].
- **Popularity** scores o by $\sum w(\Delta)$ over all past $(\cdot, r, o)$ events, the relaxed recurrence of [9].
- **Backoff** orders by recurrence, then the typed dyad, then popularity.

Figure 2 gives the first answer. On both ICEWS benchmarks about half of the test queries are recurrent, a quarter are dyad-only and a quarter are cold. On YAGO and WIKI the dyad-only stratum is 0.1%: facts there persist for years and the pair’s history is the triple’s history. The diagnostic therefore makes a prediction before any model exists: a mechanism that reads the dyad should help on ICEWS and should not help on YAGO or WIKI. Section 7 tests it.

Table 1 gives the second answer. Allowing the whole dyad to speak lifts the untrained score on ICEWS18 from MRR 28.0 to 31.8 and H@10 from 41.4 to 49.9, and on ICEWS14s from 37.1 to 43.3. The ICEWS18 figure is the published level of DaeMon and the ICEWS14s figure that of TLogic, neither of which is a weak model. On the dyad-only stratum alone, where recurrence scores exactly

Table 1: Untrained counters. Time-aware filtered, ties averaged, $\times 100$. ICEWS18 on a random sample of 30 000 test queries, ICEWS14s on all. The only learned quantity is a relation-to-relation confidence counted on the training split. Published DaeMon on ICEWS18: 31.85 / 22.67 / 49.80 [5]; TLogic on ICEWS14: 43.04 / 33.56 / 61.23 [24].

dataset	counter	MRR	H@1	H@10
ICEWS18	recurrence	28.02	20.93	41.41
	blind dyad	24.55	14.71	45.10
	typed dyad	30.65	21.05	49.82
	popularity	12.09	6.64	23.13
	backoff	31.82	22.55	49.88
ICEWS14s	recurrence	37.08	30.80	48.50
	blind dyad	36.78	26.43	57.31
	typed dyad	42.72	33.73	60.39
	popularity	15.50	8.21	30.19
	backoff	43.26	34.08	60.85

zero, the blind dyad counter reaches H@1 9.5 and H@10 42.3 on ICEWS18 and H@10 61.8 on ICEWS14s.

Two things the counters do *not* show are worth stating, because the model is designed around them. First, on the dyad-only stratum the typed counter is no better than the blind one (ICEWS18 MRR 20.33 against 20.01): the gain of the dyad comes from reading the pair at all, and whether relation types carry more than an additive confidence can extract is left to a learned sequence model. Second, the cold stratum is reached by none of the counters. A further measurement (`diagnose_paths.py`) shows that 54% of cold answers on ICEWS18 are partners of one of the subject’s 96 most recent partners, inside a set of about 555 entities, which is 2.4% of the vocabulary; a hand-built two-hop path counter combined with popularity reaches MRR 4.5 on that stratum against 2.9 for popularity alone. The path intensity of Section 5 is built for those queries.

5 ECHO

5.1 A superposition of intensities

We model the object of a query $q = (s, r, ?, t)$ as the mark of the first event of a superposition of marked point processes. Superposition adds intensities, so if $\lambda_k(o | q)$ is the intensity under which o would be emitted by process k , the probability that the answer is o is proportional to $\sum_k \lambda_k(o | q)$, and in log space the branches combine through

$$\log \lambda(o | q) = \text{logaddexp} \left(\log \lambda_{\text{struct}}(o | q), \log \lambda_{\text{pop}}(o | q), \right. \\ \left. \log \lambda_{\text{dyad}}(o | q), \log \lambda_{\text{path}}(o | q) \right). \quad (1)$$

There is no mixing weight and no gate. A learned gate between a copy branch and a neural branch is the usual design in this literature, and in our own earlier attempts it either collapsed onto one branch or oscillated between

epochs; a sum of intensities has neither failure mode, and a branch that is uninformative for a query simply contributes a small intensity. Each branch has its own learned bias, initialised at -2 , so that all four start at comparable scale. The model is trained with cross-entropy over all N entities on $\text{softmax}_o \log \lambda(o | q)$, with label smoothing, plus a small auxiliary cross-entropy on the structural branch alone that keeps it from being starved by the easier branches during the first epochs.

5.2 Support

The dyadic and path intensities act on a *support* $\mathcal{S}(q) \subset \mathcal{E}$ built from the subject’s past alone: the S_2 objects that most recently occurred with (s, r) and the S_1 entities with which s most recently interacted under any relation, in either direction, de-duplicated. With $S_1 = 96$ and $S_2 = 64$ the support covers every recurrent and dyad-only answer of ICEWS18 except those of hub subjects with more partners than the budget. Everything the two intensities read about a candidate is computed from events strictly before t by vectorised prefix operations over the subject’s event list, once per timestamp and shared by all queries at that timestamp; Section 6 reports the cost.

5.3 Dyadic intensity

For each candidate $o \in \mathcal{S}(q)$ the last L events of the dyad stream $\mathcal{D}_{so}(t)$, most recent first, become tokens

$$\mathbf{x}_i = \mathbf{E}_\rho[\rho_i] + \phi(\delta_i), \quad i = 1, \dots, L, \quad (2)$$

where $\rho_i \in \{1, \dots, 2|\mathcal{R}|\}$ is the typed, directed relation of the i -th event and $\delta_i = t - \tau_i \geq 1$ its elapsed time in snapshots. The time encoding is

$$\phi(\delta) = \mathbf{W}_\phi \left[\frac{1}{4}\ell, \sin(\omega_1\ell), \cos(\omega_1\ell), \right. \\ \left. \dots, \sin(\omega_8\ell), \cos(\omega_8\ell) \right], \quad \ell = \log(1 + \delta), \quad (3)$$

with ω_k geometrically spaced in $[1, 24]$. Nothing in Eq. (3) is monotone in δ , so the intensity built on it can peak at any elapsed time the data asks for, which a decay kernel cannot. A read-out token $\mathbf{q} = \mathbf{E}_q[r] + \mathbf{W}_s \mathbf{E}_t[s]$ carries the query relation and the evolved subject state; the sequence $[\mathbf{q}; \mathbf{x}_1; \dots; \mathbf{x}_L]$ passes through two pre-norm Transformer blocks with four heads and dimension 64, and the output at the read-out position is the stream state $\mathbf{z}_{so}$. One mechanism therefore covers self-excitation (past r events on the dyad, which is recurrence), cross-excitation (past $\rho \neq r$ events raising r), reciprocity (events in the other direction, which carry the inverse type) and order (the sequence $\rho_1\rho_2$ differs from $\rho_2\rho_1$).

Truncating the stream to L events must not lose counts, so the state is concatenated with eight summary statistics of the full history, $\mathbf{a}_{so} = [\log(1 + n_{sro}), \log(1 + \delta_{sro}), \log(1 + \bar{g}_{sro}), \mathbf{1}[n_{sro} > 0], \log(1 + n_{so}), \log(1 + \delta_{so}), \log(1 + \bar{g}_{so}), n_{sro}/n_{so}]$, where n , δ and $\bar{g}$ are the count, the recency and the mean inter-arrival gap of the triple

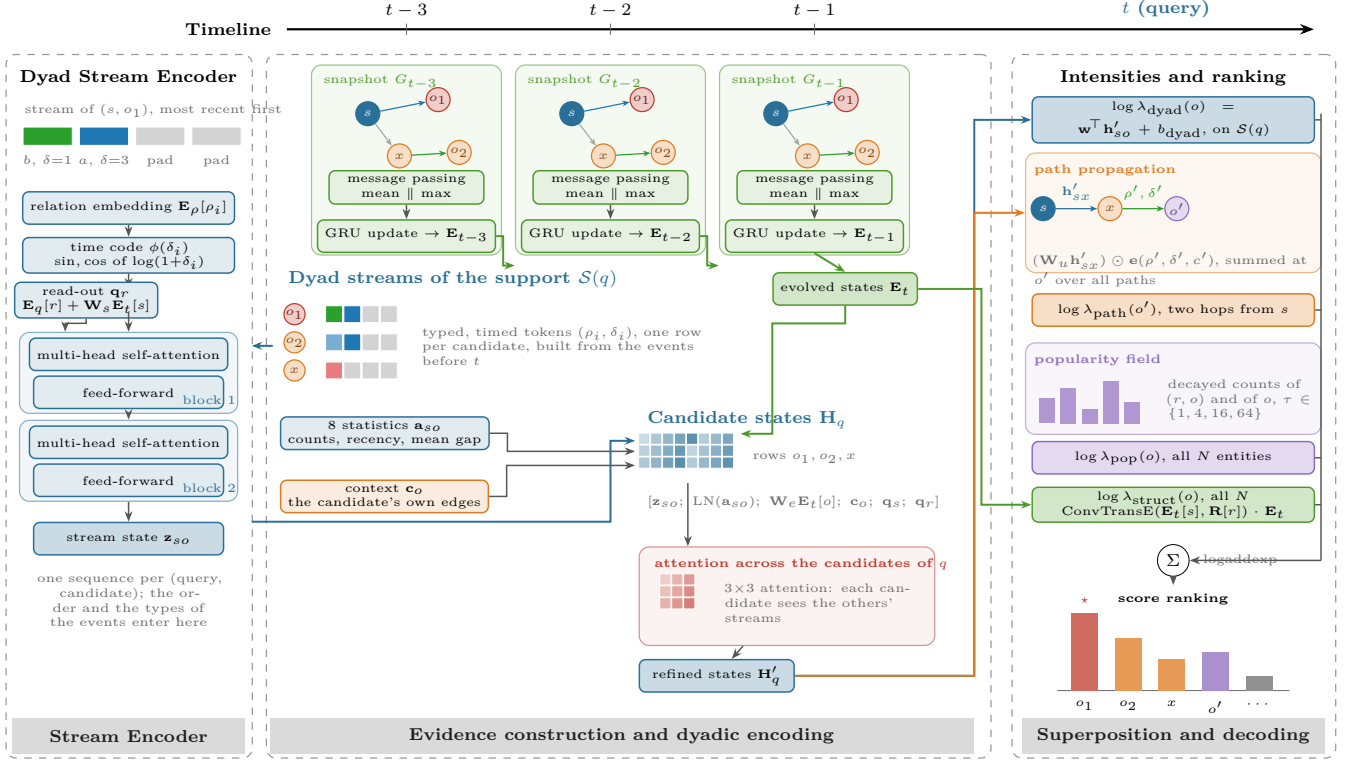


Figure 3: Architecture of ECHO. *Middle:* for the query $(s, r, ?, t)$ the structural branch evolves entity states through the snapshots before t (message passing, then a GRU update per snapshot) into $\mathbf{E}_t$; in parallel the support of s is read as dyad streams, one row of typed, timed tokens per candidate, together with eight summary statistics and the candidate’s own recent edges; the per-candidate states are assembled into $\mathbf{H}_q$, pass through one attention layer across the candidates of the query, and become $\mathbf{H}'_q$. *Left:* the stream encoder, enlarged: relation embedding plus sinusoidal time code per token, the query relation as read-out token, two Transformer blocks, read-out $\mathbf{z}_{so}$. *Right:* the four intensities. The dyadic intensity is a linear read-out of $\mathbf{H}'_q$ on the support; the path intensity forwards each refined state along the candidate’s own typed edges and sums the messages arriving at second-hop entities; the popularity intensity scores every entity from multi-scale decayed counts; the structural intensity decodes $\mathbf{E}_t$ with ConvTransE. The four log-intensities are combined by logaddexp and the model is trained with one cross-entropy over all entities.

(s, r, o) and of the dyad (s, o) . The candidate is also described by what *it* has been doing with anyone: its P_2 most recent distinct typed edges (ρ', o', δ', c') , each embedded as $\mathbf{E}_{\rho'} + \phi(\delta') + \mathbf{W}_c \log(1 + c')$, are mean-pooled into a context vector $\mathbf{c}_o$. The per-candidate state is then

$$\mathbf{h}_{so} = \text{MLP}([\mathbf{z}_{so}; \text{LN}(\mathbf{a}_{so}); \mathbf{W}_e \mathbf{E}_t[o]; \mathbf{c}_o; \mathbf{q}_s; \mathbf{q}_r]), \quad (4)$$

with $\mathbf{q}_s = \mathbf{W}'_s \mathbf{E}_t[s]$ and $\mathbf{q}_r = \mathbf{W}_r \mathbf{R}[r]$.

Competition. A scorer that looks at one candidate at a time cannot express “the answer rotates among these partners” or “the partner that has gone quiet is due”. The states of all candidates of one query therefore pass through one further attention block *across the support*, masked to the valid candidates, so that $\mathbf{h}'_{so}$ depends on the streams of the other candidates. The dyadic log-intensity is a linear read-out, $\log \lambda_{\text{dyad}}(o | q) = \mathbf{w}^\top \mathbf{h}'_{so} + b_{\text{dyad}}$, and is $-\infty$ outside the support. On the yearly datasets, where a query rarely has more than one candidate with a live triple history, the competition block is switched off; Section 6 states how that choice was made.

5.4 Path intensity

The dyad is silent whenever s and the answer never interacted, which is a quarter of the ICEWS queries. The path intensity extends the reach of the dyad by one hop. Each first-hop candidate $x \in \mathcal{S}(q)$ forwards its query-conditioned state along its own P_2 most recent typed edges $(x, \rho', o', \delta', c')$:

$$\begin{aligned} \mathbf{m}_{x \rightarrow o'} &= (\mathbf{W}_u \mathbf{h}'_{sx}) \odot \mathbf{e}(\rho', \delta', c'), \\ \mathbf{e}(\rho', \delta', c') &= \text{MLP}(\mathbf{E}'_{\rho'} + \phi(\delta') + \mathbf{W}'_c \log(1 + c')), \end{aligned} \quad (5)$$

and the messages arriving at each second-hop entity o' are summed, $\tilde{\mathbf{m}}_{o'} = n_{o'}^{-1/2} \sum_x \mathbf{m}_{x \rightarrow o'}$ over the $n_{o'}$ paths that reach it, excluding paths back to s . The path log-intensity is

$$\log \lambda_{\text{path}}(o' | q) = \text{MLP}([\text{LN}(\tilde{\mathbf{m}}_{o'}); \log(1 + n_{o'}); \mathbf{W}_e \mathbf{E}_t[o']; \mathbf{q}_s; \mathbf{q}_r]) + b_{\text{path}}, \quad (6)$$

and $-\infty$ for entities no path reaches. A path is thus the composition of a full event stream, read by the Trans-

former, with a typed, timed edge: a differentiable length-two temporal rule whose body is a sequence rather than a single past fact. Because every first-hop state is already computed, the path costs one sparse gather and one scatter-add per chunk of queries.

5.5 Popularity and structural intensities

The popularity intensity scores *every* entity from evidence rather than from an embedding alone. For a timestamp t and the relations present in its queries, decayed counts of $(\cdot, r, o)$ events at scales $\tau \in \{1, 4, 16, 64\}$ snapshots and an undecayed total, and the same five counts of o under any relation, are computed on the device by one scatter over the events before t . The ten log-counts pass through a 16-unit layer modulated by the relation embedding and a linear read-out, giving $\log \lambda_{\text{pop}}(o | q)$ for all o . This is the relaxed-recurrency signal of [9] in learned, multi-scale form, without a top- k cut.

The structural intensity is the backbone of this literature and is not a contribution. Entity states $\mathbf{E}_t$ are obtained by evolving the static table $\mathbf{E}_0$ through the H snapshots before t with a relational message-passing layer that keeps mean and max aggregates and links each layer to the same depth at the previous snapshot, a GRU cell across snapshots, and a gate back to $\mathbf{E}_0$; a ConvTransE decoder with LayerNorm scores $\mathbf{E}_t[s]$ and $\mathbf{R}[r]$ against all of $\mathbf{E}_t$. LayerNorm replaces the customary BatchNorm because batches are snapshots of very uneven size that are further chunked, and batch statistics would make the objective depend on the chunking. Trained alone this branch reaches the published level of RE-GCN (Section 7).

5.6 Training

Training iterates over timestamps. For a timestamp t the structural states and the popularity table are computed once, the graph is cut there, and the queries at t are processed in chunks bounded by a budget on (query, candidate) pairs; each chunk’s loss is backpropagated to the detached states, whose accumulated gradients are then carried once through the evolver and the popularity head. The gradient is identical to that of a single backward pass and memory is bounded by one chunk. Supports, streams and second-hop edges are built by vectorised prefix operations over the subject’s and the candidates’ event lists, in data-loader workers, with nothing later than $t - 1$ ever read; a released script deletes every event at or after t and checks bit-for-bit that nothing the model reads at t changes.

6 Experiments

6.1 Setup

Datasets. We use the five standard benchmarks in the versions released with RE-GCN and CEN [22, 20]:

ICEWS14s (7 128 entities, 230 relations, 365 daily snapshots), ICEWS18 (23 033 / 256 / 304), GDELT (7 691 / 240 / 2 975 fifteen-minute snapshots), YAGO (10 623 / 10 / 189 yearly) and WIKI (12 554 / 24 / 232 yearly), with the usual chronological 80/10/10 split. No auxiliary data is used: in particular, we do not use the static entity-type graph that RE-GCN and its descendants optionally add on ICEWS.

Baselines. The comparison in Table 2 quotes each number from the paper that produced it and names the source, because numbers produced by different harnesses are not interchangeable [11]. Rows for CyGNet, xERTE, TITer, RE-GCN, CEN, DaeMon, TiPNN and DiMNet on ICEWS18 and GDELT come from the DiMNet table [5]; rows on YAGO and WIKI from the DaeMon table [4]; LogCL, HisRES, CID-TKG and CHE-TKG from their own papers; the diffusion rows from NADEX and FreqDiff; CognTKE, the recurrency baseline and CountTRuCoLa from their own papers. We additionally re-ran LogCL, the strongest competitor with released code, under our protocol on ICEWS18; it reaches MRR 36.0 there, consistent with its published 35.67. Methods marked † have not released code at the time of writing and could not be re-run.

Implementation. Entity and relation dimension 200, two message-passing layers, history $H = 10$ snapshots (5 on GDELT), stream length $L = 10$ (16 on YAGO and WIKI, 8 on GDELT), stream dimension 64 with two blocks of four heads, support $S_1 = 96$, $S_2 = 64$ (48 on YAGO and WIKI), second-hop edges $P_2 = 16$, path dimension 32. AdamW with learning rate 10^{-3} , weight decay 10^{-3} , 5% linear warm-up then cosine to $2 \cdot 10^{-5}$, at most 20 epochs with early stopping after 6 epochs without validation improvement, gradient clipping at 1, label smoothing 0.1 (0.05 on YAGO and WIKI), dropout 0.25 on event data and 0.15 on yearly data, structural auxiliary weight 0.3 (0 on YAGO and WIKI). The competition block is on for the event datasets and off for YAGO and WIKI; validation MRR makes that choice on all five datasets (for it: ICEWS18 36.92 vs 36.70, ICEWS14s 48.99 vs 48.81, GDELT 27.45 vs 27.38; against it: YAGO 87.41 vs 87.14, WIKI 83.38 vs 83.29), and both test results are reported in Table 3. Training runs in bfloat16 on one A100-40GB; an ICEWS18 epoch takes about 3 minutes and a run about 50 minutes, GDELT about 15 minutes per epoch. Every number is the mean of three seeds (1, 2, 3) with its standard deviation.

6.2 Main results

Table 2 reports the comparison. We read it in three parts and state each exactly.

YAGO and WIKI. ECHO is above every published number we found on MRR and H@1 on both yearly benchmarks, including DaeMon, which held them for three years,

Table 2: Main results, time-aware filtered, $\times 100$. Each baseline number is quoted from the paper named in the source column; ECHO is the mean of three seeds (standard deviations in Table 3). †: no released code. Best in **bold**, second underlined. ICEWS14s rows other than DiMNet, LogCL, NADEX and ECHO are from the CID-TKG table.

method	source	ICEWS18			ICEWS14s			GDELT			YAGO			WIKI		
		MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10	MRR	H@1	H@10
Recurrency baseline	[9]	28.7	—	—	—	—	—	24.5	—	—	90.9	—	—	81.5	—	—
CyGNet	[5, 4]	24.93	15.90	42.61	—	—	—	18.48	11.52	31.98	52.07	45.36	63.77	33.89	29.06	41.86
xERTE	[5, 4]	29.31	21.03	45.60	—	—	—	18.09	12.30	30.34	84.19	80.09	89.78	71.14	68.05	79.01
TITer	[5, 4]	29.98	22.05	44.83	—	—	—	15.46	10.98	24.31	87.47	84.89	90.27	75.50	72.96	79.02
RE-GCN	[5, 4, 18]	30.58	21.01	48.75	42.39	31.96	62.29	19.64	12.42	33.69	84.12	80.76	89.98	77.55	73.75	83.68
CEN	[5]	30.84	21.23	49.67	—	—	—	20.18	12.84	34.10	—	—	—	—	—	—
TiRGN	[18]	33.66	23.19	54.22	44.75	34.26	65.28	21.67	13.63	37.60	—	—	—	—	—	—
DaeMon	[5, 4]	31.85	22.67	49.80	—	—	—	20.73	13.65	34.23	<u>91.59</u>	<u>90.03</u>	93.34	82.38	78.26	88.01
TiPNN	[5]	32.17	22.74	50.72	—	—	—	21.17	14.03	34.76	—	—	—	—	—	—
CountTRuCoLa	[10]	32.8	—	—	—	—	—	23.8	—	—	90.9	—	—	82.7	—	—
DiMNet	[5]	34.13	23.29	55.80	45.72	34.41	67.99	21.93	14.03	37.49	—	—	—	—	—	—
CognTKE	[3]	35.24	—	—	—	—	—	—	—	—	—	—	—	<u>83.21</u>	—	—
LogCL	[2]	35.67	24.53	57.74	48.87	37.76	70.26	23.75	14.64	42.33	—	—	—	—	—	—
DiffuTKG	[8]	35.65	25.19	59.55	47.58	36.38	66.01	21.35	14.43	36.05	—	—	—	—	—	—
NADEX	[7]	36.84	27.58	60.58	49.03	39.45	70.55	23.67	16.10	43.05	—	—	—	—	—	—
FreqDiff†	[8]	36.85	26.85	<u>61.47</u>	50.85	39.82	71.48	25.04	17.64	41.83	—	—	—	—	—	—
HisRES†	[31]	37.69	26.46	59.70	50.48	39.57	71.09	26.58	16.90	<u>46.31</u>	—	—	—	—	—	—
CHE-TKG†	[17]	<u>38.77</u>	<u>27.58</u>	60.95	<u>51.91</u>	<u>41.37</u>	<u>72.17</u>	<u>27.38</u>	<u>17.73</u>	47.02	—	—	—	—	—	—
CID-TKG†	[18]	38.88	27.66	61.16	51.93	41.49	72.32	27.41	<u>17.76</u>	47.03	—	—	—	—	—	—
ECHO (ours)		36.54	26.36	56.21	47.43	37.49	66.16	27.10	18.16	44.62	91.82	90.57	<u>93.23</u>	83.41	80.44	<u>87.39</u>

and CognTKE’s WIKI MRR. On H@10 DaeMon remains ahead by 0.1 and 0.6 points. The three-seed spread is 0.20 MRR on YAGO and 0.12 on WIKI, so the H@1 margins of 0.54 and 2.18 are well outside it.

GDELT. ECHO has the best H@1 of any published method, 18.16 against 17.76 for CID-TKG, and the second MRR, 0.3 behind CID-TKG and above HisRES and every method with code. The seed spread is 0.02.

ICEWS18 and ICEWS14s. ECHO is above every method with released code, including LogCL both as published and as re-run under our protocol, DiMNet, NADEX and the diffusion models on ICEWS18 MRR. It is below HisRES, CHE-TKG and CID-TKG, which report 1.2 to 2.3 more MRR on ICEWS18 and 3 to 4.5 more on ICEWS14s, and none of which has released code. We do not claim those gaps are protocol artefacts; we note only that gaps of this size are within what differences in evaluation settings have been shown to produce in this field [11], and that the one member of the family we could re-run reproduced its published number. The H@10 gap to this family (4 to 6 points) is larger than the MRR gap and is where ECHO is weakest; Section 7 locates it in the cold strata.

Competition. Table 3 isolates the one per-dataset switch. Attention across the candidates of a query adds 0.25 MRR on ICEWS18, 0.25 on ICEWS14s and 0.09 on GDELT, in the same direction on every seed, and removes 0.13 and 0.20 on WIKI and YAGO. The diagnostic predicts this: on the yearly datasets a query rarely has more than one candidate with a live triple history, so there is nothing to compete, and the extra block only adds

Table 3: Competition across candidates, test MRR / H@1, three seeds. Validation MRR selects the setting in bold on every dataset; the unselected setting is reported for completeness.

dataset	without competition	with competition
ICEWS18	36.29 $\pm$ 0.04 / 26.10	36.54 $\pm$ 0.10 / 26.36
ICEWS14s	47.18 $\pm$ 0.04 / 37.19	47.43 $\pm$ 0.14 / 37.49
GDELT	27.01 $\pm$ 0.01 / 18.05	27.10 $\pm$ 0.02 / 18.16
WIKI	83.41 $\pm$ 0.12 / 80.44	83.28 $\pm$ 0.17 / 80.22
YAGO	91.82 $\pm$ 0.20 / 90.57	91.62 $\pm$ 0.12 / 90.33

variance. Validation MRR makes the same choice as test on all five datasets, which is why the switch is a dataset setting rather than a tuned result.

Training behaviour. Every run selects its checkpoint between epochs 5 and 12 of 20, and the structural branch alone needs 17. A model that reads the history directly has little to memorise. Ten alternative recipes on ICEWS18 (16- and 20-epoch schedules, dropout 0.35 and 0.45, weight decay 10^{-3} , learning rate $5 \cdot 10^{-4}$, auxiliary weight 0 and 1, label smoothing 0.2, wider supports) all land between 36.0 and 36.3 MRR and all peak at epochs 8 to 10; the plateau is a property of what the model can see, not of regularisation, and the ablations of Section 7 say which parts of what it sees matter.

Table 4: Performance by stratum on ICEWS18 and ICEWS14s, final model, three seeds. Strata partition the test set, so the rows weight to the aggregates of Table 2.

stratum	ICEWS18				ICEWS14s			
	share	MRR	H@1	H@10	share	MRR	H@1	H@10
cold-far	17.8%	1.75	0.57	2.98	15.9%	2.19	0.53	4.42
cold-2hop	14.0%	9.43	3.40	20.67	12.0%	11.28	3.38	27.92
dyad-only	19.2%	29.82	17.06	57.26	21.2%	37.54	23.44	67.20
blocked	25.7%	34.95	16.76	72.55	16.6%	38.44	17.65	79.68
clean	23.4%	86.43	77.87	99.07	34.4%	90.50	84.18	99.33

7 Analysis

The aggregate numbers of Table 2 say how much; this section says where and why. Every stratum is defined from the data alone (Definition 1) and partitions the test set, so the rows of each table weight to the aggregate.

7.1 Where the queries are, and where the errors are

Table 4 is the central table of the paper. On ICEWS18 the clean stratum, where the answer recurred for (s, r) and dominates every distractor on recency and count, is a quarter of the queries and is essentially solved (H@10 99.0). The blocked stratum, the same size, is where a monotone copy score provably cannot order the answer first; ECHO ranks it first 16.8% of the time and in the top ten 72.6% of the time, so the ordering information it uses is not count-and-recency. The dyad-only stratum, a fifth of the queries and invisible to every copy mechanism, reaches MRR 29.8 and H@10 57.3: the untrained blind-dyad counter reached 20.0 and 42.3 on the same stratum (Section 4), and the learned stream adds ten points on top of it. The two cold strata, a third of the queries, are where the model is weak: an answer that is a partner of a partner is found in the top ten one time in five, and an answer that is further than two hops from the subject is almost never found. The H@10 gap to the global-graph family in Table 2 is of the order of the mass in these two rows, and we take it to be where the next step lies.

7.2 Ablations

Table 5 removes one part at a time on ICEWS18, where the dyad-only stratum is 19% of the queries, and on YAGO, where it is 0.1%. Three readings follow.

The dyadic branch is the model. Without it ECHO is a snapshot encoder with a popularity prior and scores 30.30 on ICEWS18, which is RE-GCN’s published 30.58, and 67.18 on YAGO. The branch is worth 6.0 MRR on the event dataset and 24.6 on the yearly one.

The stream acts where the diagnostic said it would. On ICEWS18 the Transformer over the event stream is worth 0.58 MRR over the eight summary statistics alone, and the relation types inside it are worth 0.43:

Table 5: Ablations on an event dataset and a yearly dataset: MRR and its change relative to the model without competition. Rows marked ³ are three-seed means (spread at most 0.19); the others are seed 1. The seed spread of the base is 0.04 on ICEWS18 and 0.20 on YAGO. H@1 moves with MRR in every row (ICEWS18 base 26.10, without the dyadic branch 20.51; YAGO base 90.57, without it 61.37).

removed	ICEWS18		YAGO	
	MRR	Δ	MRR	Δ
nothing (base) ³	36.29		91.82	
popularity field	36.37	+0.08	91.81	−0.01
candidate context	36.27	−0.02	91.75 ³	−0.07
path intensity	35.92	−0.37	91.70 ³	−0.12
relation types in the stream	35.86	−0.43	91.79	−0.03
the stream (statistics only)	35.71	−0.58	91.83	+0.01
structural branch	35.59	−0.70	90.97	−0.85
dyadic branch	30.30	−5.99	67.18	−24.64
added competition ³	36.54	+0.25	91.62	−0.20

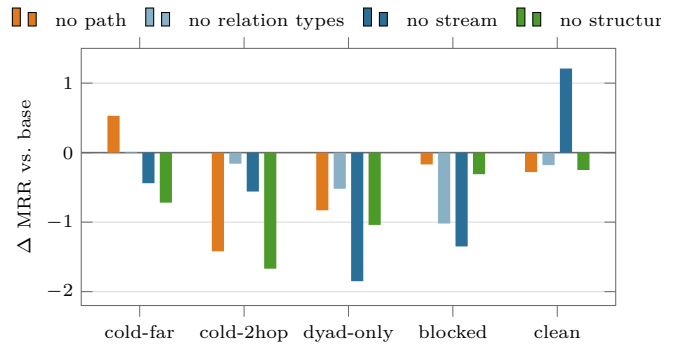


Figure 4: Where each part acts (ICEWS18, seed 1, change of MRR per stratum when the part is removed). The path intensity is paid for on cold-2hop; the relation types on blocked; the stream on dyad-only and blocked; the structural branch on both cold strata. Removing the whole dyadic branch costs 3.5 on dyad-only, 5.1 on blocked and 17.4 on clean, off the scale of this plot.

a stream with types replaced by a single placeholder loses most of what the stream adds. The path intensity adds 0.37, and Table 4 locates it: on cold-2hop queries it lifts MRR from 6.21 to 8.52 and H@10 from 13.4 to 19.0 on seed 1. The seed spread on ICEWS18 is 0.04, so each of these effects is an order of magnitude above noise. On YAGO the same three parts are worth +0.01, −0.03 and −0.12, all inside the 0.20 seed spread. The eight statistics carry the dyadic branch on their own there, because on yearly data the question is only whether a fact is still running, and the count and recency of the triple answer it. This is the prediction made in Section 4 before any model was trained, and it is the kind of evidence a stratum-level analysis can give and a leaderboard cannot.

Figure 4 resolves the same ablations by stratum, and each part is paid for where it was built: the path intensity on cold-2hop (−1.42 MRR when removed), the relation

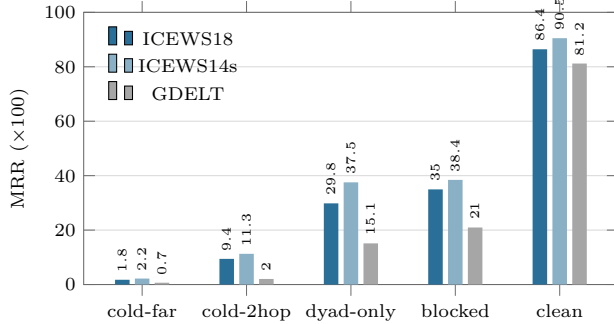


Figure 5: MRR by stratum on the three event datasets (three seeds). The profile is the same on all three: the clean stratum is solved, the blocked and dyad-only strata are where a copy mechanism fails and where ECHO earns its margin, and the two cold strata are where the remaining headroom lies. GDELT, whose fifteen-minute snapshots make every stratum sparser, is lower everywhere but keeps the ordering.

types on the blocked stratum (-1.02), the stream on dyad-only and blocked (-1.85 and -1.35), and the structural branch on the two cold strata (-0.72 and -1.67), which the dyad cannot see at all. No part except the dyadic branch itself costs the clean stratum more than 0.3 points. The parts of a superposition are additive in the strata they touch, which is what a sum of intensities should look like and what a gated mixture typically does not.

Two parts are null and are reported as such. The popularity field and the candidate-context input move neither dataset outside the seed spread; a prototype intensity that scores every entity by similarity to the query’s past answers, tried for the cold-far stratum, is likewise null ($+0.05$ on ICEWS18, -0.16 ± 0.19 on YAGO) and is kept only as an ablation switch. We leave them in the released code with their switches rather than silently removing them, so that the ablation table is reproducible as printed.

7.3 The same strata on every event dataset

Figure 5 repeats the stratified view on ICEWS14s and GDELT. The profile is identical across the three event datasets, which is what one expects of strata defined from the data rather than from the model: the clean stratum sits above 80 MRR everywhere, the blocked and dyad-only strata in the 15 to 40 range, and the cold strata near zero. The same figure says why GDELT is the hardest benchmark in the suite despite its small vocabulary: at fifteen-minute resolution its dyad streams are sparse, and every stratum except the clean one is a third to a half lower than on ICEWS.

8 Conclusion

We asked what unit of evidence a temporal knowledge graph forecaster should read, and measured the answer before modelling it. The dyad, the ordered entity pair with the typed stream of every event between its members, carries information that the query pair does not: a quarter of ICEWS queries have an answer that only the dyad can see, and a counter that reads the dyad with no training matches a published model that reads the query pair with a great deal of it. ECHO turns that measurement into a model: a superposition of intensities, with a Transformer over the dyad’s event stream at its centre, a path intensity that composes a stream with a typed edge, and attention across the candidates of a query. Over three seeds it sets the best published MRR and H@1 on YAGO and WIKI, the best H@1 on GDELT, and exceeds every method with released code on ICEWS18 and ICEWS14s. The stratified analysis shows each intensity acting where the data said it would, and shows as plainly where the model does not act: a third of ICEWS queries whose answer never interacted with the subject remain largely out of reach, and three recent methods without released code report higher aggregate numbers on those benchmarks. Reaching the cold strata, and settling the comparison with those methods under one protocol, are the two open ends of this work. The code, the diagnostic and every result file are released so that both can be taken up.

References

- [1] Yuxiang Cai, Qiao Liu, Yanglei Gan, Changlin Li, Xueyi Liu, Run Lin, Da Luo, and Jiaye Yang. Predicting the unpredictable: Uncertainty-aware reasoning over temporal knowledge graphs via diffusion process. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5766–5778, 2024.
- [2] Wei Chen, Huaiyu Wan, Yuting Wu, Shuyuan Zhao, Jiayao Cheng, Yuxin Li, and Youfang Lin. Local-global history-aware contrastive learning for temporal knowledge graph reasoning. In *IEEE 40th International Conference on Data Engineering (ICDE)*, 2024. arXiv:2312.01601.
- [3] Wei Chen, Yuting Wu, Shuhan Wu, Zhiyu Zhang, Mengqi Liao, Youfang Lin, and Huaiyu Wan. Cogn-TKE: A cognitive temporal knowledge extrapolation framework. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025. arXiv:2412.16557.
- [4] Hao Dong, Zhiyuan Ning, Pengyang Wang, Ziyue Qiao, Pengfei Wang, Yuanchun Zhou, and Yanjie Fu. Adaptive path-memory network for temporal knowledge graph reasoning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence (IJCAI)*, 2023. arXiv:2304.12604.

- [5] Hao Dong, Ziyue Qiao, Zhiyuan Ning, Qi Hao, Yi Du, Pengyang Wang, and Yuanchun Zhou. Disentangled multi-span evolutionary network against temporal knowledge graph reasoning. In *Findings of the Association for Computational Linguistics: ACL 2025*, 2025. arXiv:2505.14020.
- [6] Hao Dong, Pengyang Wang, Meng Xiao, Zhiyuan Ning, Pengfei Wang, and Yuanchun Zhou. Temporal inductive path neural network for temporal knowledge graph reasoning. *Artificial Intelligence*, 2024. arXiv:2309.03251.
- [7] Yanglei Gan, Peng He, Yuxiang Cai, Run Lin, Guanyu Zhou, and Qiao Liu. Negative-aware diffusion process for temporal knowledge graph extrapolation. In *Findings of the Association for Computational Linguistics: EACL 2026*, 2026. arXiv:2602.08815.
- [8] Yanglei Gan, Peng He, Run Lin, Peiyuan Jiang, Yifan Wang, and Qiao Liu. Denoising the future: Context-aware spectral diffusion for temporal knowledge graph extrapolation. *arXiv preprint arXiv:2608.20804*, 2026.
- [9] Julia Gastinger, Christian Meilicke, Federico Errica, Timo Sztyler, Anett Schuelke, and Heiner Stuckenschmidt. History repeats itself: A baseline for temporal knowledge graph forecasting. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, 2024. arXiv:2404.16726.
- [10] Julia Gastinger, Christian Meilicke, and Heiner Stuckenschmidt. CountTRuCoLa: Rule learning for interpretable temporal knowledge graph forecasting. In *The 25th International Semantic Web Conference (ISWC)*, 2026. arXiv:2509.09474.
- [11] Julia Gastinger, Timo Sztyler, Lokesh Sharma, Anett Schuelke, and Heiner Stuckenschmidt. Comparing apples and oranges? on the evaluation of methods for temporal knowledge graph forecasting. In *Machine Learning and Knowledge Discovery in Databases: Research Track (ECML PKDD)*, 2023.
- [12] Zhen Han, Peng Chen, Yunpu Ma, and Volker Tresp. xERTE: Explainable reasoning on temporal knowledge graphs for forecasting future links. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2012.15537.
- [13] Zhen Han, Yunpu Ma, Yuyi Wang, Stephan Günnemann, and Volker Tresp. Graph hawkes neural network for forecasting on temporal knowledge graphs. In *Automated Knowledge Base Construction (AKBC)*, 2020. arXiv:2003.13432.
- [14] Rikui Huang, Shengzhe Zhang, and Wei Wei. Strikingness-aware evaluation for temporal knowledge graph reasoning. *arXiv preprint arXiv:2605.13153*, 2026.
- [15] Woojeong Jin, Meng Qu, Xisen Jin, and Xiang Ren. Recurrent event network: Autoregressive structure inference over temporal knowledge graphs. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020. arXiv:1904.05530.
- [16] Seunghan Lee, Jun Seo, Jaehoon Lee, Sungdong Yoo, Minjae Kim, Tae Yoon Lim, Dongwan Kang, Hwanil Choi, SoonYoung Lee, and Wonbin Ahn. AdaTKG: Adaptive memory for temporal knowledge graph reasoning. *arXiv preprint arXiv:2605.07121*, 2026.
- [17] Shuai-Long Lei, Xiaobin Zhu, Jiarui Liang, Guoxi Sun, Zhiyu Fang, and Xu-Cheng Yin. CHE-TKG: Collaborative historical evidence and evolutionary dynamics learning for temporal knowledge graph reasoning. *arXiv preprint arXiv:2605.04652*, 2026.
- [18] Shuai-Long Lei, Xiaobin Zhu, Jiarui Liang, Guoxi Sun, Zhiyu Fang, and Xu-Cheng Yin. CID-TKG: Collaborative historical invariance and evolutionary dynamics learning for temporal knowledge graph reasoning. *arXiv preprint arXiv:2604.09600*, 2026.
- [19] Yujia Li, Shiliang Sun, and Jing Zhao. TiRGN: Time-guided recurrent graph network with local-global historical patterns for temporal knowledge graph reasoning. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.
- [20] Zixuan Li, Saiping Guan, Xiaolong Jin, Weihua Peng, Yajuan Lyu, Yong Zhu, Long Bai, Wei Li, Jiafeng Guo, and Xueqi Cheng. Complex evolutionary pattern learning for temporal knowledge graph reasoning. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2022.
- [21] Zixuan Li, Zhongni Hou, Saiping Guan, Xiaolong Jin, Weihua Peng, Long Bai, Yajuan Lyu, Wei Li, Jiafeng Guo, and Xueqi Cheng. HiSMATCH: Historical structure matching based temporal knowledge graph reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 2022. arXiv:2210.09708.
- [22] Zixuan Li, Xiaolong Jin, Wei Li, Saiping Guan, Jiafeng Guo, Huawei Shen, Yuanzhuo Wang, and Xueqi Cheng. Temporal knowledge graph reasoning based on evolutionary representation learning. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2021. arXiv:2104.10353.
- [23] Yansong Liu, Rui Liu, Yuan Zuo, Hongwei Zhao, Da Fu, Fuwei Zhang, Fuzhen Zhuang, Yong Chen, and Zhe Li. HiTS-CL: A continual learning framework for long-horizon temporal knowledge graph extrapolation. *arXiv preprint arXiv:2609.36559*, 2026.

- [24] Yushan Liu, Yunpu Ma, Marcel Hildebrandt, Mitchell Joblin, and Volker Tresp. TLogic: Temporal logical rules for explainable link forecasting on temporal knowledge graphs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2022. arXiv:2112.08025.
- [25] Konrad Özdemir, Julia Gastinger, Lukas Kirchdorfer, and Heiner Stuckenschmidt. Temporal knowledge graph forecasting under distribution shifts: A synthetic evaluation. *arXiv preprint arXiv:2607.09232*, 2026.
- [26] Haohai Sun, Jialun Zhong, Yunpu Ma, Zhen Han, and Kun He. TimeTraveler: Reinforcement learning for temporal knowledge graph forecasting. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021. arXiv:2109.04101.
- [27] Xiangni Tian, Kaixian Yu, Rungpeng Dai, Niansheng Tang, and Hongtu Zhu. GAttNHP: Group attention neural hawkes process for extrapolation reasoning in temporal knowledge graphs. *arXiv preprint arXiv:2607.14733*, 2026.
- [28] Rakshit Trivedi, Hanjun Dai, Yichen Wang, and Le Song. Know-evolve: Deep temporal reasoning for dynamic knowledge graphs. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017. arXiv:1705.05742.
- [29] Yi Xu, Junjie Ou, Hui Xu, and Luoyi Fu. Temporal knowledge graph reasoning with historical contrastive learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023. arXiv:2211.10904.
- [30] Jiasheng Zhang, Zhangpin Li, Mingzhe Wang, Jie Shao, Jiangtao Cui, and Hui Li. Towards better evolution modeling for temporal knowledge graphs. *arXiv preprint arXiv:2602.08353*, 2026.
- [31] Jinchuan Zhang, Bei Hui, Chong Mu, Ming Sun, and Ling Tian. Historically relevant event structuring for temporal knowledge graph reasoning. *arXiv preprint arXiv:2405.10621*, 2024. Stated by the authors as accepted to ICDE 2025.
- [32] Ze Zhao, Yuhui He, Lyuwen Wu, Gu Tang, Bin Lu, Xiaoying Gan, Luoyi Fu, Xinbing Wang, and Chenghu Zhou. Inductive reasoning for temporal knowledge graphs with emerging entities. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2604.10164.
- [33] Cunchao Zhu, Muhao Chen, Changjun Fan, Guangquan Cheng, and Yan Zhan. Learning from history: Modeling temporal knowledge graphs with sequential copy-generation networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021. arXiv:2012.08492.